Task 1 - ECC Boot and SPDM Baseline

PSoC Control C3M6 (KIT_PSC3M6_EVAL) - reference configuration before the PQC migration

1. Summary

The supplied ECC-based boot and SPDM implementation was built, deployed and measured on the evaluation kit. All positive tests passed and every negative (tamper) test was rejected. Boot-time protection is provided by the Edge Protect Bootloader (EPBL) verifying the application with **ECDSA-P384**; real-time protection is SPDM 1.4 profile **P2** (ECDSA-P384 authentication, ECDHE-P384 key exchange, SHA-384, AES-256-GCM). These numbers are the reference for the PQC (ML-DSA-87 / ML-KEM-1024) comparison in Tasks 2-3.

Area	Result	Key figures
Boot-time (EPBL, ECDSA-P384)	2/2 positive PASS, 6/6 negative rejected	verify ~170 ms; reset-to-app ~266 ms; EPBL 35.9 KB flash
Real-time (SPDM P2)	9/9 positive PASS, 1/1 negative rejected	sign 84.4 ms; session setup ~257 ms device time; cert chain 1,218 B
Lifecycle / ownership	DEVELOPMENT, Infineon default owner	no OEM key provisioned, default SIMPLE_APP boot policy; nothing irreversible performed

2. Test setup

Item	Configuration
MCU / kit	PSC3M6GES3AH (Cortex-M33, TrustZone), KIT_PSC3M6_EVAL, 512 KB flash, 128 KB SRAM
Lifecycle state	DEVELOPMENT (read via J-Link: ROM Boot 1.0.0.13973, Flash Boot 91.191.28)
Toolchain	ModusToolbox 3.9, GCC 14.2.1, Edge Protect Security Suite 2.4.0, J-Link V9.76a
Programming	On-board J-Link OB (USB J13)
Host link (SPDM / provisioning)	KitProg3 (fw 2.90.1804, kp3-bulk) as USB-I2C bridge, I2C 0x08 @ 400 kHz, DFU host tool dfuh-cli; MCTP framing, <=127 B SPDM per DFU transfer
Bootloader	EPBL (CE43373), validation ECDSA-384, validate_boot + validate_upgrade + sw_downgrade_prev = true, overwrite memory map
Boot test app	CE243041 secure-boot app, MCUboot format (header 0x400), signed with ECDSA-P384 sample key
SPDM responder	psoc-spdm-responder, SPDM_PROFILE=P2, CONFIG=Release (-Os) + measurement logging
SPDM requester	spdm-requester.exe 2.1.0.0 (Windows host)
SPDM identity	ECDSA-P384 key generated on-device in PAS slot 0; chain (demo CA ca_ec384 + leaf) in PAS slot 1
Measurement method	Device: NS SysTick cycle counter around each SPDM dispatch and each secure-side crypto call ([PERF] lines on UART); stack painting; heap break high-water. Host: wall clock per requester command.

3. Boot chain (baseline)

```
Reset
 +- ROM_BOOT   (immutable, v1.0.0.13973)          classical root of trust, no ML-DSA
     +- FLASH_BOOT (v91.191.28)                   applies lifecycle + boot policy
         +- EPBL @0x32000000 (MCUboot based)      NOT signature-checked in this setup
             +- verifies primary slot @0x3201B000 ECDSA-P384 (baseline) -> ML-DSA-87 (Task 2)
                 +- application
```

Memory map: EPBL 0x32000000 (108 KB), primary slot 0x3201B000 (128 KB), upgrade slot 0x32040000 (128 KB). The SPDM responder uses its own map (secure 128 KB at 0x32000000, non-secure 160 KB at 0x32020000, credential store PAS 24 KB at 0x32048000); integrating it behind EPBL is part of Task 2.

Platform behaviour (quoted from the application notes):

Source	Statement
AN243370 sec. 4.1	'BootROM consists of ROM_BOOT and FLASH_BOOT.' Without authentication: 'ROM_BOOT applies basic trim, configures clocks, and validates the SFlash ... FLASH_BOOT launches the next stage application (OEM) without authentication.' With secure boot: 'FLASH_BOOT authenticates the next stage application (OEM) ... If authentication fails, the device will go to the DEAD branch.'
AN243370 Table 9	Chain: ROM (immutable) -> ASSET_HASH (eFuse) -> SFLASH_BOOT code -> OEM_ROT_KEY (checked against eFUSE_HASH) -> OEM application (checked with OEM_ROT_KEY).

Source	Statement
AN243370 sec. 1.3	BootROM secure boot is 'ECDSA (P-256, P-384, or P-521)'. 'a device in its factory state does not enforce image authentication and boots any image.' PQC requires 'Edge Protect Bootloader (EPB) ... as first OEM application in the boot chain.'
AN243370 sec. 2.2.2	'Infineon ships the device with development LCS and provisioned with SIMPLE_APP configuration ... launches the application without authentication.'
AN243370 sec. 2.1.1/2.2.1/2.2.6	Development LCS: 'Device can be provisioned as many times as needed ... device can be safely recovered'. Re-provisioning: with prov_oem_complete = false 'the device can be re-provisioned any number of times - in either LCS'. Production: 'Unlike in development LCS, the device ownership becomes permanent and cannot be transferred to a different owner'. Only the LCS transition (eFuse) is irreversible.
AN243376 sec. 2.1	Taking ownership needs an OEM certificate containing OEM_ROT_PUBLIC_KEY, 'signed by the IFX_ROT_KEY, and generated by Infineon's Edge Protect Signing Service (EPSS)'.

OEM root-of-trust key - status on this device: no OEM key is provisioned; the device is in DEVELOPMENT LCS under Infineon default ownership with the factory SIMPLE_APP boot policy (edgeprotecttools device-info: silicon 0xF100, family 0x124, rev 0x11, LCS DEVELOPMENT). Evidence that FLASH_BOOT does not authenticate the next stage: the EPBL image at 0x32000000 is a raw image (first word = initial stack pointer 0x3400E800, no MCUboot header 0x96F3B83D, no signature) and it boots. **Nothing anchors EPBL today**; the chain of trust starts at EPBL.

Who verifies what (baseline): ROM_BOOT validates SFLASH/FLASH_BOOT (Infineon, immutable); FLASH_BOOT launches EPBL unauthenticated; EPBL authenticates the application (SHA-384 image hash, key hash, ECDSA-P384 signature over header + image + protected TLVs) on every boot and before every update; the image version in the protected header is used for downgrade prevention.

Closing the gap: enabling BootROM authentication of EPBL is possible in DEVELOPMENT LCS and reversible (restore to Infineon default policy), but requires an Infineon-signed OEM development certificate from EPSS (OSTS account), then provisioning boot_cfg_id = SECURE_APP with an **ECDSA** OEM_ROT_KEY and an EPBL signed in MCUboot format (CE243041 docs, section 3). Even then the BootROM-to-EPBL step remains ECDSA - a classical dependency that ML-DSA in EPBL cannot remove.

4. Boot-time protection results (EPBL, ECDSA-P384)

#	Test	Expected	Observed (EPBL log)	Result
B1	Authorized image v1.0.0 signed with trusted ECDSA-P384 key	boot	ECDSA verify OK, 'User Application validated successfully', app prints 1.0.0+0	PASS
B2	Corrupted image: 1 byte flipped at 0x3201C000	reject	'IMAGE_TLV_SHA is invalid', 'Image in the primary slot is not valid!', app not started	REJECTED
B3	Incorrectly signed: valid build signed with rogue ECDSA-P384 key	reject	key-hash TLV mismatch (returned 0x2e), 'not valid', app not started	REJECTED
B4	Unauthorized rollback: correctly signed v0.9.0 placed in upgrade slot (primary = v1.0.0)	reject	EPBL log: '[0s.47ms][ERR] insufficient version in secondary slot'. Flash readback BEFORE boot: upgrade slot header 96F3B83D, version bytes 00 09 (0.9). AFTER boot: upgrade slot erased (all 0x00), primary unchanged (version 01), app prints 1.0.0+0	REJECTED
B5	Valid update v2.0.0+1 placed in upgrade slot	install	verified, copied secondary->primary (128 KB), re-verified, app prints 2.0.0+1	PASS
B6	Modified MCUboot header metadata: image version byte @0x3201B014 changed	reject	'IMAGE_TLV_SHA is invalid' (header is covered by the image hash), app not started	REJECTED
B7	Modified protected TLV metadata @0x3202546C	reject	'IMAGE_TLV_SHA is invalid', app not started	REJECTED
B8	Modified signature TLV (byte inside ECDSA signature @0x32025585)	reject	hash OK, ECDSA verification fails, 'Image in the primary slot is not valid!'	REJECTED

Image layout used for B6-B8 (tools/mcuboot_info.py): header 0x3201B000 (0x400 B), payload 0xA064 B, protected TLVs @0x32025464 (137 B), TLV area @0x320254ED: SHA-384 (48 B), KEYHASH (48 B), ECDSA_SIG (103 B, DER). B4 log re-captured (boot_ECDSA-384_rollback_attempt.log): image load, readback and reset now run in one J-Link session while the UART is already open, so the complete first boot after the load is recorded; the earlier capture had missed it.

Boot metric	ECDSA-P384 baseline	Source
Image signature verification in EPBL	~170 ms	EPBL log timestamps 59 ms -> 229 ms
Reset to application start (no update)	~266 ms	EPBL log 'Starting User Application'
Reset to application start with update install	~3.8 s	EPBL log 3s.802: erase (~2.5 s) + copy 128 KB + 2 verifications
EPBL flash (.text + data)	35,888 B	EPBL build memory report
EPBL static RAM (.bss) / heap region	6,504 B / 31,080 B	EPBL build memory report
Signature material in image (TLVs)	96 B signature + 48 B key hash + 48 B SHA-384	MCUboot TLVs (ML-DSA-87 signature will be 4,627 B)

5. Real-time protection results (SPDM 1.4, profile P2)

Negotiated: SPDM 1.4, ECDSA-P384 (base asym), SHA-384, ECDHE secp384r1, AES-256-GCM. Responder = PSoC, requester = Windows host, trust anchor pinned to ca_ec384.

#	Requester operation	What it proves	Host wall clock	Result
S1	connect	VERSION / CAPABILITIES / ALGORITHMS negotiation	5.4 s	PASS
S2	slots	GET_DIGESTS, slot 0 populated	6.6 s	PASS
S3	get-cert	1,218 B chain fetched and verified to pinned root	26.5 s	PASS
S4	challenge	CHALLENGE_AUTH: responder proves possession of ECDSA-P384 key	29.7 s	PASS
S5	measure	signed GET_MEASUREMENTS	33.6 s	PASS
S6	session	KEY_EXCHANGE (ECDHE-P384) / FINISH / END_SESSION	38.3 s	PASS
S7-9	exchange x3	encrypted + authenticated app message (AES-256-GCM), echoed correctly	39-44 s	PASS
N1	challenge with wrong root (ca_ec256)	invalid credential / trust anchor must be rejected	33.0 s	REJECTED (SPDM_REQ_EAUTH)

5.1 Device-side timing (from [PERF], stable within +/-0.2 % across repeated runs)

Operation (on PSoC)	Time	Notes
ECDSA-P384 sign (device identity, secure side)	84.4 ms	software mbedTLS (only P-256 is HW accelerated)
ECDHE-P384 ephemeral key generation	80.7 ms	secure side
ECDHE-P384 shared secret	82.2 ms	secure side
CHALLENGE_AUTH message (total)	85.3 ms	dominated by the signature
GET_MEASUREMENTS signed (total)	85.1 ms	
KEY_EXCHANGE message (sign + keygen + shared secret)	252.2 ms	
FINISH (secured)	5.0 ms	HMAC / key schedule
Session setup, device time (KEY_EXCHANGE + FINISH)	~257 ms	
Unsigned SPDM messages (VERSION, CAPS, ALGS, DIGESTS)	0.25 - 0.73 ms	
CHUNK_GET (128 B fragment)	0.31 ms	
Encrypted application message (AEAD decrypt + encrypt)	1.15 - 1.23 ms	

5.2 Communication

Metric	P2 baseline
Largest SPDM message	CERTIFICATE response 1,226 B -> 10 x CHUNK_GET (128 B DFU payloads)
Large signed responses	CHALLENGE_AUTH 190 B, MEASUREMENTS 295 B (sent via chunking)
KEY_EXCHANGE	request 166 B (2 x CHUNK_SEND), response KEY_EXCHANGE_RSP 302 B (3 x CHUNK_GET)
FINISH / END_SESSION (secured)	FINISH 86 B request / 60 B response; END_SESSION 34 B / 37 B (wire size incl. MCTP + secured-message header)
Full 'session' command	24 DFU transfers, 2,569 B on the wire (VERSION ... certificate fetch ... KEY_EXCHANGE, FINISH, END_SESSION)
Host time per transfer	~1.4 s (dfuh-cli process start per transaction) - host-bound, not device-bound
Application latency	~38 s per 'exchange' command end-to-end on this host stack; device processing per encrypted message ~1.2 ms

6. Memory

6.1 Static footprint, uninstrumented Release builds

Region	P2 ECC baseline	P4 PQC (preview)	Delta
Secure flash (of 128 KB)	82,300 B	79,532 B	-2,768 B
Non-secure flash (of 160 KB)	48,036 B	47,972 B	-64 B
Total responder flash	130,336 B	127,504 B	-2,832 B
Secure .bss	11,016 B	18,940 B	+7,924 B
Non-secure .bss (libspdm buffers)	19,952 B	57,816 B	+37,864 B
SPDM certificate chain (PAS slot 1, 15 KB)	1,218 B	15,168 B	+13,950 B
Bootloader EPBL flash	35,888 B	(Task 2)	

P-384 runs in software mbedTLS, so the ECC build is not smaller in flash than the wolfCrypt ML-DSA/ML-KEM build; the PQC cost shows up in RAM buffers and credential size. The SRAM regions report 100 % in the build because the heap is sized to fill the remainder.

6.2 Runtime peaks (P2, worst case over all operations)

Region	Peak used	Available	Method
Non-secure stack	2,328 B	4,096 B	stack painting
Non-secure heap	1,032 B	44,600 B	heap break high-water
Secure stack	4,336 B	8,192 B	stack painting (read via NSC veneer)
Secure heap	2,576 B	39,040 B	_sbrk high-water (read via NSC veneer)

7. Limitations and remarks

- BootROM/FLASH_BOOT do not authenticate EPBL (no OEM key, SIMPLE_APP). Enabling it is reversible in DEVELOPMENT LCS but needs an Infineon EPSS-signed OEM certificate; the BootROM step would still be ECDSA (remaining classical dependency). Must stay visible in the final presentation.
- Still open for the validation matrix (planned in Task 3 for both P2 and P4 in one session): modified responder certificate

stored on the device, modified handshake message (responder fault-injection build).
- Boot tests used the CE243041 demo application; the SPDM responder is not yet behind EPBL (different memory map; part of Task 2).
- The update image was placed in the upgrade slot with J-Link instead of the DFU transport; EPBL processing is identical.
- Demo keys/CA from the code examples were used; not production credentials.
- DWT->CYCCNT reads 0 from the non-secure world on this device; timings use the NS SysTick counter at SystemCoreClock (measurement code adds ~16 KB NS flash, logging only).
- Host wall-clock numbers are dominated by the DFU host tool (~1.4 s per transfer) and are not representative of a production BMC requester.
- Hardware notes: KitProg3 bridge needed firmware 2.90 + bulk mode for I2C; the rollback test overwrote the PAS region, so the SPDM identity must be re-provisioned before Task 3.

8. Evidence (files in results/)

File	Content
task1_P2_20261111_1215/	requester output per operation, device_uart.log ([PERF]), summary.txt
task1_provision_p2.log	ECDSA-P384 provisioning incl. prove-cert
resp_p2_release.log / resp_p4_release.log	static memory reports
boot_ECDSA-384_v1_0.log, boot_ecdsa384_NEG_corrupt.log, boot_ecdsa384_NEG_wrongkey.log, boot_ECDSA-384_v0_9_rollback.log, boot_ECDSA-384_v2_0_valid_update.log	EPBL boot logs (B1-B5)
neg_corrupt_payload.hex, neg_rogue_key.hex, img_ECDSA-384_*.hex	exact images used in the boot tests
boot_evidence/	B4 boot log + flash readback BEFORE/AFTER boot (summary_ECDSA-384_rerun_rollback.txt), B6-B8 tampered images + boot logs, summary
doc_text/AN243370.txt, AN243376.txt	text of the application notes used for the quotes
TASK1_NOTES.md	working notes with the same data